AI sistem za proceduralno generisanje tekstura iz realnih fotografija

Predlog istraživačko-razvojnog projekta sa primenom u igranju u oblaku i infrastrukturi centara podataka

Autor: Milan Majstorović

Datum: 19. oktobar 2026.

Sadržaj

Apstrakt 4
1. Uvod 5
2. Predmet, problem i motivacija 6
2.1 Istraživačko pitanje 6
2.2 Radne hipoteze 6
3. Pregled oblasti 7
3.1 Klasična i neuronska kompresija tekstura 7
3.2 Proceduralne teksture 7
3.3 Programska sinteza i diferencijabilno renderovanje 7
3.4 Istraživački jaz 7
4. Ciljevi i doprinos projekta 8
4.1 Originalni doprinos 8
5. Predložena arhitektura sistema 9
5.1 Priprema ulaza 9
5.2 Vizuelni enkoder 9
5.3 Generator proceduralnog grafa 9
5.4 Optimizator parametara 9
5.5 Hibridni rezidualni sloj 9
5.6 Generator i verifikator shader koda 9
6. Metodologija istraživanja 10
6.1 Skup podataka 10
6.2 Bazne metode 10
6.3 Funkcija cilja 10
6.4 Metrike 11
6.5 Hardversko i softversko okruženje 12
6.6 Eksperimentalni postupak 12
6.7 Statistička obrada i reproduktivnost 13
6.8 Kriterijumi uspeha 13

7. Primena u igranju u oblaku 14
7.1 Potencijalne koristi 14
7.2 Režimi izvršavanja i prethodno generisanje tekstura 14
7.3 Važno ograničenje 15
7.4 Adaptivna politika 16
8. Šire implikacije za infrastrukturu data centara 17
8.1 Skladištenje, replikacija i keširanje 17
8.2 Raspoređivanje računskih zadataka 17
8.3 Energetska i ugljenična efikasnost 17
8.4 Primene izvan igara 17
8.5 Budući pravci: fotonske i hibridne arhitekture 18
9. Rizici i ograničenja 19
10. Etički, pravni i ekološki aspekti 20
10.1 Podaci, autorska prava i licence 20
10.2 Validacija i bezbednost AI-generisanog koda 20
10.3 Transparentnost i reproduktivnost 20
10.4 Ekološka procena 20
11. Plan istraživanja i razvoja 21
12. Očekivani rezultati 21
13. Zaključak 22
14. Literatura 23

Apstrakt

Ovaj predlog istraživačko-razvojnog projekta predstavlja sistem koji primenom veštačke inteligencije pretvara realne fotografije u kompaktne proceduralne reprezentacije i izvršivi *shader* kod. Cilj nije samo smanjenje veličine teksturnih resursa, već optimizacija ukupnog odnosa između memorije, skladištenja, mrežnog prenosa, vremena učitavanja i troška izvršavanja. Cloud gaming (igranje u oblaku) je izabran kao glavna oblast primene jer omogućava centralizovano upravljanje resursima i merenje efekata na velikom broju instanci. Šire implikacije za data centre razmatraju se kao zasebno poglavlje, pošto obuhvataju raspoređivanje radnih opterećenja, keširanje, replikaciju podataka i energetsku efikasnost. Predložena metodologija uključuje izgradnju skupa podataka, sintezu proceduralnih programa, generisanje shader koda, višekriterijumsku optimizaciju i eksperimentalno poređenje sa klasičnim i neuronskim pristupima.

Ključne reči: proceduralne teksture, veštačka inteligencija, programska sinteza, *shader* (program za grafičko senčenje), kompresija, igranje u oblaku, centri podataka.

1. Uvod

Savremene interaktivne aplikacije koriste veliki broj tekstura visoke rezolucije, zbog čega rastu zahtevi za skladištenjem, memorijom i propusnim opsegom. Klasični blokovski formati optimizovani su za brzo dekodiranje na grafičkom procesoru (GPU), dok neuronske metode pokušavaju da smanje otisak uz dekodiranje na zahtev [1], [9]. Proceduralna reprezentacija koristi matematičke funkcije i mali skup parametara kako bi sadržaj rekonstruisala tokom izvršavanja.

Proceduralno senčenje i funkcije šuma dugo se koriste za kompaktno stvaranje vizuelne složenosti [4]–[6]. Njihovo ručno projektovanje, međutim, zahteva ekspertsko znanje, dok dodatno računanje može povećati opterećenje procesora i grafičkog procesora. Novija istraživanja pokazuju da se grafovi proceduralnih materijala mogu automatski generisati i optimizovati na osnovu slike ili teksta [2], [3].

U ovom radu igranje u oblaku nije naknadna primena, već jedan od osnovnih scenarija evaluacije. Platforme u oblaku centralizuju renderovanje i upravljanje resursima, ali računarska snaga i dalje utiče na cenu, potrošnju energije, gustinu korisnika i kašnjenje [7], [8]. Zbog toga projekat posmatra proceduralno generisanje kao merljiv kompromis između veličine podataka i troška izvršavanja, a ne kao bezuslovnu zamenu za klasične teksture.

2. Predmet, problem i motivacija

Predmet istraživanja je automatsko pronalaženje proceduralnog programa koji vizuelno aproksimira ulaznu fotografiju ili teksturni uzorak. Program treba da bude dovoljno kompaktan, prenosiv i efikasan da se izvršava u izabranom grafičkom okruženju.

Centralni problem može se formulisati kao višekriterijumska optimizacija: potrebno je istovremeno povećati perceptivnu sličnost sa originalom i smanjiti veličinu programa, vreme generisanja, vreme izvršavanja, memorijsku potrošnju i količinu podataka koja se skladišti ili prenosi. Ovi ciljevi mogu biti suprotstavljeni, pa sistem mora da pronađe skup kompromisnih rešenja umesto jednog univerzalnog optimuma.

2.1 Istraživačko pitanje

Da li AI sistem može automatski da generiše proceduralnu reprezentaciju realne teksture koja, pri prihvatljivom nivou vizuelnog kvaliteta, smanjuje ukupan infrastrukturni trošak u odnosu na klasično skladištenje i isporuku tekstura?

2.2 Radne hipoteze

1. Za teksture sa izraženom stohastičkom, periodičnom ili višeskalnom strukturom moguće je postići manju reprezentaciju uz prihvatljiv perceptivni gubitak.
2. Hibridni pristup, koji kombinuje proceduralne funkcije i male rezidualne teksture, biće stabilniji od potpuno proceduralnog pristupa na složenim fotografijama.
3. Dobit u skladištenju i I/O operacijama neće automatski značiti niži ukupni trošak; rezultat će zavisiti od učestalosti ponovne upotrebe, keširanja i cene izvršavanja na CPU-u/GPU-u.
4. U okruženju igranja u oblaku adaptivni izbor između klasične, proceduralne i hibridne reprezentacije može poboljšati iskorišćenje infrastrukture.

3. Pregled oblasti

3.1 Klasična i neuronska kompresija tekstura

Klasični formati kompresije čuvaju diskretne blokove ili transformisane koeficijente slike i optimizovani su za brzo dekodiranje. Neuronske metode uče latentnu reprezentaciju i koriste mali dekoder za rekonstrukciju; primer predstavlja neuronska kompresija tekstura (NTC) sa nasumičnim pristupom i dekodiranjem u realnom vremenu [1]. Obe grupe mogu ostvariti visok kvalitet, ali ne proizvode nužno čitljiv proceduralni program niti omogućavaju jednostavno semantičko menjanje strukture teksture.

3.2 Proceduralne teksture

Proceduralne teksture nastaju kombinovanjem gradijenata, funkcija šuma, fraktalnih kompozicija, ćelijskih struktura, transformacija koordinata, filtriranja i mešanja slojeva. Perlinovi, talasni (*wavelet*) i Gaborovi pristupi ilustruju različite odnose između brzine, spektralne kontrole i artefakata [4]–[6]. Prednosti proceduralne reprezentacije su kompaktnost, skalabilna rezolucija i parametrizacija, dok su nedostaci trošak izvršavanja i ograničena sposobnost reprodukovanja semantički složenih fotografija.

3.3 Programska sinteza i diferencijabilno renderovanje

Problem se može posmatrati kao sinteza programa: sistem bira graf operatora i njegove parametre tako da renderovani izlaz odgovara ciljnoj slici. Postojeći radovi pokazuju generisanje proceduralnih grafova iz slike ili teksta i gradijentnu optimizaciju grafova koji sadrže nediferencijabilne operatore [2], [3]. Ovaj projekat proširuje taj pravac infrastrukturnim ciljevima i izvozom u izvršivi *shader* kod.

3.4 Istraživački jaz

U ograničenom skupu pregledanih izvora postojeća literatura zasebno obrađuje neuronsku kompresiju tekstura [1], generisanje i optimizaciju proceduralnih grafova [2], [3], proceduralne funkcije šuma [4]–[6] i infrastrukturu igranja u oblaku [7]. Na osnovu ovog preliminarnog pregleda nije identifikovan jedinstven sistem koji iz realne fotografije generiše prenosiv program za grafičko senčenje, dopušta hibridni rezidualni sloj i optimizuje rešenje prema kvalitetu, veličini, vremenu izvršavanja, memoriji, I/O opterećenju i energiji. Taj spoj predstavlja potencijalni istraživački jaz koji projekat cilja; tvrdnja o originalnosti može se potvrditi tek nakon dokumentovane sistematske pretrage literature.

4. Ciljevi i doprinos projekta

1. Definisati biblioteku proceduralnih operatora i ograničenu gramatiku programa.
2. Razviti model za analizu vizuelnih obrazaca i predlaganje strukture proceduralnog grafa.
3. Generisati validan kôd u jezicima GLSL, HLSL ili WGSL iz izabrane reprezentacije.
4. Optimizovati kvalitet, veličinu, vreme izvršavanja i infrastrukturni trošak.
5. Razviti hibridnu reprezentaciju za delove slike koji se ne mogu efikasno opisati procedurama.
6. Eksperimentalno oceniti korist u lokalnom okruženju i okruženju igranja u oblaku.
7. Formulisati smernice za širu primenu u data centrima.

4.1 Originalni doprinos

Predloženi doprinos nije nova pojedinačna funkcija šuma niti još jedan neuronski kodek. Originalnost se zasniva na integraciji četiri elementa:

1. **Infrastrukturno svesna programska sinteza:** proceduralni graf se ne optimizuje samo prema slici, već i prema merljivom trošku izvršavanja i isporuke.
2. **Adaptivni izbor reprezentacije i režima isporuke:** sistem za svaki resurs bira klasičnu, potpuno proceduralnu ili hibridnu proceduralno-rezidualnu reprezentaciju, a zatim određuje da li će se ona izvršavati neposredno, prethodno generisati i keširati ili koristiti u hibridnom režimu.
3. Paretoov izlaz: umesto jednog rešenja, sistem proizvodi skup varijanti za različite procesorske, memorijske i energetske budžete.
4. Evaluacioni protokol za infrastrukturu oblaka: kvalitet teksture povezuje se sa vremenom pokretanja instance, gustinom sesija, I/O operacijama, keširanjem i potrošnjom energije.

U odnosu na generisanje proceduralnih materijala [2] i optimizaciju grafova [3], projekat dodaje verifikaciju programa za grafičko senčenje i infrastrukturnu funkciju cilja. U odnosu na neuronsku kompresiju [1], rezultat je eksplicitni proceduralni program ili hibridni opis koji se može profilisati i menjati.

5. Predložena arhitektura sistema

5.1 Priprema ulaza

Ulazne slike se normalizuju po rezoluciji i prostoru boja, a zatim se analiziraju radi utvrđivanja periodičnosti, frekvencijskog sadržaja, ivica, regiona, dominantnih boja i višeskalnih struktura. Za materijale se, kada su dostupne, zajedno obrađuju mape osnovne boje, normala, hrapavosti i drugih svojstava.

5.2 Vizuelni enkoder

Enkoder proizvodi skup karakteristika koje opisuju lokalne i globalne obrasce. Njegov izlaz ne mora neposredno da rekonstruiše sliku; njegova uloga je da usmeri izbor proceduralnih operatora, početnih parametara i podele na regione.

5.3 Generator proceduralnog grafa

Generator bira operatore iz kontrolisane biblioteke i povezuje ih u usmereni graf. Ograničena gramatika sprečava nevalidne kombinacije, ograničava složenost i olakšava izvoz u više jezika programa za grafičko senčenje. Kandidati se rangiraju prema funkciji cilja koja kombinuje perceptivni kvalitet i trošak.

5.4 Optimizator parametara

Parametri frekvencije, skale, amplituda, boja, pragova i mešanja iterativno se prilagođavaju ciljnoj slici. Za svakog kandidata beleže se kvalitet rekonstrukcije, broj operacija, veličina koda, vreme kompajliranja i vreme izvršavanja.

5.5 Hibridni rezidualni sloj

Kada proceduralni graf ne može ekonomično da reprodukuje određeni detalj, razlika se čuva kao mala rezidualna tekstura ili skup lokalnih zakrpa. Time se omogućava kontrolisana razmena između čistoće proceduralnog pristupa i robusnosti klasične reprezentacije.

5.6 Generator i verifikator shader koda

Završni graf se prevodi u ciljni *shader* jezik. Verifikator proverava sintaksu, numeričku stabilnost, podržane instrukcije, granice resursa i konzistentnost rezultata na izabranim platformama.

6. Metodologija istraživanja

6.1 Skup podataka

Planirani skup obuhvata **1.200 uzoraka** u četiri uravnotežene klase: 300 stohastičkih površina, 300 periodičnih obrazaca, 300 prirodnih materijala i 300 semantički složenih scena. Svaki uzorak priprema se u rezolucijama 512 × 512, 1.024 × 1.024 i, kada izvor dozvoljava, 2.048 × 2.048 piksela. Podela je 70% za razvoj i obuku, 15% za validaciju i 15% za završno testiranje, uz grupisanje po izvornom materijalu kako varijante iste površine ne bi završile u različitim skupovima. Beleže se izvor, licenca, rezolucija, klasa, prostor boja i dostupne PBR mape. Ako licencirani skup ne dostigne planirani obim, stvarni broj i odstupanje biće eksplicitno prijavljeni.

6.2 Bazne metode

Bazne metode su: (1) nekompresovana referenca, (2) BC7 za boju i BC5 za dvokanalne mape, (3) AMD Compressonator sa dokumentovanim podešavanjima [9], (4) neuronska kompresija sa nasumičnim pristupom kada je dostupna reproduktivna implementacija [1], i (5) ručno izrađen proceduralni graf za stratifikovani podskup od najmanje 40 uzoraka. Sve metode proizvode istu ciljnu rezoluciju i ocenjuju se na istim slikama i hardveru.

6.3 Funkcija cilja

Za kandidata x definiše se normalizovana funkcija cilja: $J(x) = \alpha D(x) + \beta S(x) + \gamma T(x) + \delta M(x) + \varepsilon E(x)$, gde je D perceptivna distorzija, S veličina koda, parametara i reziduala, T medijana GPU vremena, M vršna memorija i I/O, a E energija po generisanju ili reprezentativnom ciklusu korišćenja. Svaka komponenta se deli vrednošću odgovarajuće bazne metode. Primarni eksperiment koristi jednake pondere 0,20; analiza osetljivosti koristi profile „kvalitet“, „memorija“ i „trošak infrastrukture oblaka“. Pored rangiranja po J, objavljuje se Pareto front kako rezultat ne bi zavisio samo od izabranih pondera.

6.4 Metrike

Dimenzija	Metrike	Svrha
Vizuelni kvalitet	PSNR, SSIM, perceptivna metrika i korisnička procena	Merenje vernosti i subjektivne prihvatljivosti
Kompaktnost	Veličina koda, parametara i reziduala	Poređenje skladišnog i prenosnog otiska
Izvršavanje	Vreme izvršavanja na GPU-u/CPU-u, broj instrukcija i vreme kompajliranja	Procena troška generisanja i iscrtavanja
Resursi	VRAM/RAM, I/O operacije i potrošnja energije	Procena opterećenja sistema
Cloud efekat	Vreme pokretanja instance, gustina instanci, stopa pogodaka keša i mrežni promet između čvorova	Procena infrastrukturne koristi
Režim isporuke	vreme prethodnog generisanja, veličina keširanog izlaza, VRAM, mipmap nivoi i GPU format	Poređenje neposredne, prethodno generisane i hibridne varijante

6.5 Hardversko i softversko okruženje

U planiranoj evaluaciji primarna merenja izvodiće se na jednoj unapred fiksiranoj radnoj stanici sa diskretnim GPU-om koji podržava ciljnu grafičku API specifikaciju, najmanje 8 GB VRAM-a, najmanje 16 GB sistemske memorije i SSD skladištem. Pre početka eksperimenta u radu će se upisati tačni modeli CPU-a i GPU-a, količina memorije, operativni sistem, verzija drajvera, grafički API, kompajler *shader* programa i identifikator verzije izvornog koda implementacije. Za procenu prenosivosti koristiće se najmanje još jedan GPU druge generacije ili proizvođača. Rezultati različitih uređaja neće se spajati u jedan prosek.

6.6 Eksperimentalni postupak

1. Zaključati test skup, konfiguraciju i slučajne *seed* vrednosti pre završne evaluacije.
2. Za svaki uzorak generisati najmanje pet kandidata po reprezentaciji i sačuvati najbolji validacioni kandidat bez uvida u test metrike.
3. Izvršiti deset zagrevanja i najmanje 100 merenih prolaza iscrtavanja; prijaviti medijanu i 95% interval pouzdanosti dobijen bootstrap metodom ponovnog uzorkovanja.
4. Za isti uzorak uporediti neposredno proceduralno izvršavanje, prethodno generisanje i hibridni režim. Za prethodno generisani izlaz izmeriti vreme izrade, veličinu distribuiranog opisa i keširane teksture, zauzeće VRAM-a, vreme učitavanja i kvalitet nakon pripreme mipmap nivoa i kompresije u odgovarajući GPU format.
5. Meriti hladno pokretanje instance, ponovljeno korišćenje, pogodak/promašaj keša, vršnu memoriju, čitanje sa skladišta i energiju gde hardver omogućava pouzdanu telemetriju.
6. Konstruisati Pareto skup prema perceptivnoj distorziji, veličini, vremenu izvršavanja i energiji.
7. Sprovesti ablacionu analizu bez rezidualnog sloja, bez infrastrukturnih članova funkcije cilja i bez svake glavne grupe proceduralnih operatora.
8. Za korisničku procenu koristiti najmanje 24 učesnika, nasumičan redosled i slepo poređenje parova; prijaviti metod regrutacije i saglasnost učesnika.

6.7 Statistička obrada i reproduktivnost

Za svaku metriku prijavljuju se medijana, interkvartilni raspon i 95% interval pouzdanosti. Uparena poređenja na istim teksturama koriste Vilkoksonov test sa Holmovom korekcijom za višestruka poređenja, uz prag značajnosti 0,05 i veličinu efekta. Objavljuju se konfiguracije, početne vrednosti generatora slučajnosti, skripte za merenje, verzije alata i dozvoljeni deo skupa podataka.

6.8 Kriterijumi uspeha

Primarni kriterijum za **neposredno proceduralno izvršavanje** jeste da na zaključanom test skupu smanji ukupnu veličinu distribuiranog resursa za najmanje 50% u odnosu na odgovarajući BC format, uz SSIM od najmanje 0,95 i bez povećanja medijane GPU vremena većeg od 10%. Za **prethodno generisanje** isti prag kvaliteta primenjuje se na konačni GPU format, dok se odvojeno prijavljuju veličina proceduralnog opisa, veličina kompresovanog keširanog izlaza, zauzeće VRAM-a, vreme generisanja i vreme učitavanja; uspeh zahteva najmanje 20% manje pročitanih ili repliciranih bajtova u cloud scenariju, bez smanjenja broja podržanih sesija po GPU-u većeg od 5%. Za **hibridni režim** uspeh predstavlja statistički značajno poboljšanje funkcije J u odnosu na obe relevantne bazne metode i najmanje jedna klasa tekstura na kojoj pristup pripada Pareto frontu. Pragovi su unapred definisani ciljevi projekta, a ne tvrdnja o ostvarenim rezultatima.

7. Primena u igranju u oblaku

Igranje u oblaku centralizuje izvršavanje igre, ali ne uklanja cenu računanja niti zahteve za niskim kašnjenjem [7]. Dodatne operacije programa za grafičko senčenje mogu smanjiti broj istovremenih sesija po akceleratoru ili povećati potrošnju energije. Istovremeno, kompaktni proceduralni resursi mogu smanjiti prostor za instalacione slike, vreme distribucije između regiona, I/O pri pokretanju i količinu podataka koja se mora keširati.

7.1 Potencijalne koristi

- manje slike igara i brža distribucija verzija između data centara;
- smanjeno čitanje tekstura sa skladišta pri pokretanju instance;
- bolja ponovna upotreba proceduralnih programa i parametara;
- adaptivno biranje kvaliteta prema raspoloživom budžetu grafičkog procesora;
- mogućnost centralnog profilisanja i automatske optimizacije varijanti *shader* programa.

7.2 Režimi izvršavanja i prethodno generisanje tekstura

Predloženi sistem podržava tri režima upotrebe proceduralne reprezentacije: neposredno izvršavanje tokom iscrtavanja, prethodno generisanje (*pre-bake*) i hibridni režim. Neposredno izvršavanje zadržava kompaktan program i omogućava dinamičke ili animirane materijale, ali uvodi trošak pri svakom iscrtavanju. U režimu prethodnog generisanja AI sistem proizvodi proceduralni graf i *shader* (program za grafičko senčenje), koji se izvršava jednom ili povremeno radi izrade teksture, atlasa ili skupa pločica. Dobijeni izlaz se zatim koristi kao standardni GPU resurs, pa proceduralni graf ne opterećuje svaki frejm.

Prethodno generisanje premešta trošak sa vremena igranja na fazu pripreme, distribucije ili pokretanja instance. Ono može smanjiti količinu izvornog sadržaja koji se skladišti i prenosi, ali ne uklanja memorijski otisak keširane teksture. Ako se generisani izlaz čuva nekompresovan ili u nepovoljnom rasporedu, VRAM trošak može biti jednak ili veći od troška klasične teksture. Zbog toga se izlaz mora pripremiti u odgovarajućem GPU formatu, sa mipmap nivoima, blokovskom kompresijom i rasporedom prilagođenim ciljnoj platformi.

Hibridni režim omogućava da se često korišćeni ili skupi sadržaj generiše unapred i kešira, dok se retke varijante proizvode na zahtev, a dinamički delovi izvršavaju proceduralno. Izbor režima zato treba zasnivati na profilu upotrebe, učestalosti ponovne upotrebe, raspoloživom VRAM-u, trošku generisanja, kašnjenju pri prvom pristupu i trošku distribucije, umesto na pretpostavci da je računanje u oblaku zanemarlivo.

7.2.1 Mesto i trenutak rekonstrukcije

Prethodno generisanje može se izvršiti na više tačaka životnog ciklusa resursa. Tokom izgradnje paketa teksture se proizvode pre objavljivanja i distribuiraju kao standardni resursi, čime se izbegava kašnjenje pri pokretanju, ali se zadržava puna veličina generisanih izlaza u svakom regionu. Pri regionalnoj distribuciji prenosi se kompaktan proceduralni opis, a teksture se rekonstruišu jednom u ciljnom regionu i zatim keširaju; ovaj režim može smanjiti međuregionalni prenos uz dodatni regionalni računarski trošak.

Generisanje pri pokretanju instance smanjuje potrebu za trajnim čuvanjem svih varijanti, ali povećava vreme hladnog pokretanja i zahteva kontrolu konkurentnih zahteva za CPU/GPU resursima. Generisanje na zahtev odlaže rekonstrukciju do prvog pristupa i pogodno je za retke varijante, ali uvodi kašnjenje pri promašaju keša. Zbog toga evaluacija mora zasebno meriti vreme generisanja, stopu pogodaka keša, dodatno skladište, količinu prenetih podataka i amortizovani trošak prema broju ponovnih upotreba za svaku tačku rekonstrukcije.

7.2.2 Odnos prema virtuelnom teksturisanju

Pristup ima delimičnu sličnost sa konceptima virtuelnog teksturisanja, kao što je MegaTexture, jer oba mogu organizovati sadržaj u pločice i oslanjati se na keširanje samo potrebnih delova [12]. Međutim, analogija je ograničena: virtuelno teksturisanje prvenstveno rešava adresiranje, strimovanje i rezidentnost veoma velikih unapred pripremljenih tekstura, dok ovaj projekat istražuje automatsko izvođenje proceduralnog opisa iz slike, izbor mesta rekonstrukcije i kompromis između prenosa, skladištenja, generisanja i izvršavanja. Proceduralno generisane pločice mogle bi u budućnosti da budu izvor sadržaja za sistem virtuelnog teksturisanja, ali implementacija kompletnog mehanizma strimovanja i upravljanja rezidentnošću nije osnovni predmet projekta.

7.3 Važno ograničenje

Proceduralne teksture ne smanjuju neposredno video tok koji se šalje krajnjem korisniku, jer se igra u oblaku uglavnom iscrtava i video kodira u centru podataka. Njihov glavni mrežni doprinos odnosi se na distribuciju, replikaciju i učitavanje resursa unutar infrastrukture. Uticaj na izlaznu brzinu video prenosa moguć je samo posredno i mora se zasebno eksperimentalno proveriti.

7.4 Adaptivna politika

Najpraktičniji rezultat projekta može biti kontroler koji za svaki resurs bira klasičnu kompresovanu teksturu, neposredno proceduralno izvršavanje, prethodno generisanu i keširanu teksturu ili hibridnu reprezentaciju. Odluka se donosi prema trenutnom GPU opterećenju, memorijskom pritisku, dostupnosti keša, učestalosti upotrebe, vremenu prihvatljivom za prvo generisanje i energetskom budžetu.

8. Šire implikacije za infrastrukturu data centara

Posebno poglavlje je opravdano zato što se infrastrukturne posledice razlikuju od samog scenarija igranja u oblaku. Rezultati projekta mogu doprineti sistemima u kojima se često distribuiraju ili rekonstruišu veliki vizuelni resursi, ali se zaključci ne mogu automatski preneti na sve vrste podataka.

8.1 Skladištenje, replikacija i keširanje

Kompaktni programi i parametri mogu smanjiti količinu podataka koja se replicira između regiona i čuva u više nivoa keša. Nasuprot tome, generisani izlazi mogu zahtevati privremeni prostor, pa treba meriti ceo životni ciklus resursa, uključujući verzionisanje i invalidaciju keša.

8.2 Raspoređivanje računskih zadataka

Orkestrator može unapred generisati često korišćene teksture tokom perioda nižeg opterećenja, dok ređe varijante može stvarati na zahtev. Prediktivni model može odlučivati na kom čvoru i u kom trenutku se generisanje isplati, uz ograničenja kašnjenja i raspoloživosti akceleratora.

8.3 Energetska i ugljenična efikasnost

Smanjenje skladištenja nije dovoljno za tvrdnju o održivosti. Potrebno je meriti energiju potrebnu za generisanje, ponovno generisanje, memorijski pristup, mrežni prenos i čuvanje podataka. Energetski rast centara podataka čini ovakvo sistemsko merenje relevantnim, posebno pri rastu udela akceleratorskih opterećenja [8]. Tek analiza celog životnog ciklusa može pokazati da li se dodatna energija za računanje nadoknađuje uštedama u drugim komponentama.

8.4 Primene izvan igara

Potencijalne oblasti uključuju virtuelna okruženja, digitalne blizance, simulacije, prostornu vizuelizaciju i sisteme za generisanje sintetičkih podataka. Svaka oblast zahteva zasebnu proveru kvaliteta, determinističnosti, bezbednosti i reproduktivnosti.

8.5 Budući pravci: fotonske i hibridne arhitekture

Kao dugoročna istraživačka hipoteza, proceduralna reprezentacija može dobiti dodatni značaj u budućim fotonskim i hibridnim fotonsko-elektronskim centrima podataka. Fotonski akceleratori nude veliki paralelizam i propusni opseg za određene klase operacija, ali njihov sistemski učinak može biti ograničen kretanjem podataka između elektronske memorije, pretvarača i optičkih računskih jedinica. Trošak unosa, izlaza i opto-elektronske konverzije može umanjiti ubrzanje samog optičkog računanja [10].

U takvom okruženju kompaktan proceduralni program i mali skup parametara mogli bi da zamene prenos kompletnih teksturnih mapa. Program bi se preneo do čvora ili akceleratora, a vizuelni sadržaj bi se rekonstruisao lokalno i po potrebi. Time bi se potencijalno smanjili količina podataka između memorije i fotonskog akceleratora, broj ponovljenih opto-elektronskih konverzija i pritisak na elektronsko skladištenje. Ovaj efekat nije dokazana prednost projekta, već hipoteza koja zavisi od arhitekture memorije, mesta rekonstrukcije, mogućnosti keširanja i odnosa energije računanja i konverzije. Istraživanja memorije neposredno povezane sa fotonskim računskim jedinicama pokazuju da smanjenje kretanja i konverzije podataka predstavlja relevantan projektantski cilj [11].

Provera ove hipoteze zahteva zaseban sistemski model ili budući eksperiment. Takva studija treba da modeluje propusni opseg i energiju elektronske memorije, optičkih interkonekcija, digitalno-analognog i analogno-digitalnog pretvaranja (DAC/ADC) i elektro-optičkih pretvarača, kao i trošak lokalne rekonstrukcije proceduralnog sadržaja. Poređenje bi obuhvatilo prenos klasičnih tekstura, neuronskih latentnih reprezentacija i proceduralnog koda pod jednakim zahtevima vizuelnog kvaliteta. Zbog tog dodatnog obima, fotonska arhitektura ostaje budući pravac, dok su AI proceduralne teksture, igranje u oblaku i merljivi infrastrukturni troškovi centralni predmet ovog rada.

9. Rizici i ograničenja

- **Složen sadržaj:** fotografije sa objektima, tekstom ili jedinstvenim detaljima mogu biti loš kandidat za čisto proceduralni opis.
- **Trošak izvršavanja:** manji fajl može izazvati veći ukupni računarski i energetski trošak.
- **Stabilnost:** generisani shader mora raditi dosledno na različitim drajverima i arhitekturama.
- **Vreme kompajliranja:** veliki broj shader varijanti može usporiti pokretanje ili povećati složenost keša.
- **Podaci i licence:** skup za obuku mora imati jasno poreklo i dozvolu za korišćenje.
- **Generalizacija:** rezultat na teksturama ne dokazuje efikasnost za modele, animacije ili zvuk.
- **Bezbednost:** automatski generisan kod zahteva izolaciju, validaciju i ograničenje resursa.
- **Keširanje i rekonstrukcija:** promašaji keša, istovremeni zahtevi za isti resurs i neuspešna rekonstrukcija mogu povećati kašnjenje hladnog pokretanja ili privremeno opteretiti računarske čvorove.
- **Regionalno umnožavanje:** prethodno generisani izlazi mogu se duplirati u više regiona i poništiti deo skladišne uštede ako politika zadržavanja i invalidacije nije prilagođena obrascu korišćenja.

10. Etički, pravni i ekološki aspekti

10.1 Podaci, autorska prava i licence

Skup podataka mora da sadrži samo fotografije i materijale za koje postoji dokumentovana dozvola za istraživačku obradu, treniranje modela i objavljivanje izvedenih rezultata. Za svaki izvor vodi se evidencija o licenci, autorstvu, ograničenjima redistribucije i načinu citiranja. Ako skup sadrži prepoznatljive osobe, privatne prostore ili osetljive metapodatke, takvi elementi se uklanjaju ili obrađuju samo uz odgovarajući pravni osnov i saglasnost.

10.2 Validacija i bezbednost AI-generisanog koda

Generisani *shader* kod ne sme se izvršavati bez automatske provere sintakse, ograničenja resursa i ponašanja u izolovanom okruženju. Sistem treba da ograniči trajanje izvršavanja, memorijske pristupe, veličinu programa i dozvoljeni skup operatora. Svi rezultati moraju biti označeni kao izlaz AI sistema, a neuspele, nestabilne i ručno korigovane varijante evidentirane radi reproduktivnosti.

10.3 Transparentnost i reproduktivnost

Rad treba jasno da razlikuje planirane pragove od ostvarenih rezultata, kao i izmerene vrednosti od procena i simulacija. Objavljuju se verzije modela i alata, konfiguracije, slučajne *seed* vrednosti (početne vrednosti generatora slučajnosti), pravila izbora kandidata i metod statističke obrade. Kada podaci ili kod ne mogu biti javno objavljeni, navode se razlozi i obezbeđuju opisni protokoli koji omogućavaju nezavisnu proveru u najvećoj mogućoj meri.

10.4 Ekološka procena

Tvrdnje o energetskoj ili ugljeničnoj koristi zasnivaju se na merenju celog relevantnog životnog ciklusa: pripreme i obuke modela, generisanja reprezentacije, kompajliranja, renderovanja, skladištenja i mrežnog prenosa. Smanjenje veličine fajla samo po sebi nije dokaz održivosti. Projekat treba da prijavi energiju po obrađenom uzorku i po reprezentativnom ciklusu korišćenja, a zaključke ograniči na testirane hardverske i infrastrukturne uslove.

11. Plan istraživanja i razvoja

Faza	Aktivnosti	Rezultat
1. Specifikacija i literatura	Definisanje gramatike, scenarija, metrika i baznih metoda	Protokol istraživanja
2. Podaci i prototip	Priprema skupa i implementacija proceduralne biblioteke	Reproduktivni početni sistem
3. AI sinteza	Razvoj enkodera, generatora grafa i optimizatora	Model koji proizvodi kandidate
4. *Shader* i hibridni sloj	Izvoz koda, verifikacija i rezidualna reprezentacija	Izvršivi prototip
5. Evaluacija	Poređenje kvaliteta, veličine, brzine, energije i metrika infrastrukture oblaka	Eksperimentalni rezultati
6. Demonstrator	Integracija adaptivnog kontrolera i korisničkog interfejsa	R&D demonstrator i završna dokumentacija

12. Očekivani rezultati

Pošto ovaj dokument predstavlja predlog istraživačko-razvojnog projekta, navedene vrednosti predstavljaju ciljeve koji će biti provereni tek nakon implementacije. Očekivani rezultat je funkcionalni prototip koji iz ulazne teksture proizvodi proceduralni ili hibridni opis, izvršivi *shader* i izveštaj o kompromisu kvaliteta i resursa. Planirani naučni doprinos obuhvata formulaciju višekriterijumske programske sinteze, metodologiju poređenja i analizu uslova pod kojima je proceduralna reprezentacija korisna za infrastrukturu oblaka. Procentualne uštede i statistički zaključci neće se predstavljati kao ostvareni dok ne budu izvedeni iz merenja.

13. Zaključak

Ovaj predlog projekta povezuje AI, proceduralnu grafiku i upravljanje infrastrukturnim resursima. Njegova najjača verzija nije zasnovana na pretpostavci da će procesorska snaga postati besplatna, već na adaptivnom izboru reprezentacije prema stvarnom stanju sistema. Igranje u oblaku pruža relevantan i merljiv slučaj primene, dok zasebna analiza data centara omogućava šire zaključke bez gubitka fokusa rada. Ključ uspeha biće dokazivanje da uštede u skladištenju, prenosu i I/O operacijama u određenim scenarijima nadmašuju dodatni računarski i energetski trošak.

14. Literatura

[1] K. Vaidyanathan, M. Salvi, B. Wronski, T. Akenine-Möller, P. Ebelin i A. Lefohn, „Random-Access Neural Compression of Material Textures," *ACM Transactions on Graphics*, vol. 42, br. 4, čl. 88, str. 1–25, 2023, doi: 10.1145/3592407.

[2] Y. Hu, P. Guerrero, M. Hašan, H. Rushmeier i V. Deschaintre, „Generating Procedural Materials from Text or Image Prompts," u *ACM SIGGRAPH 2023 Conference Proceedings*, 2023, doi: 10.1145/3588432.3591520.

[3] Y. Hu, P. Guerrero, M. Hašan, H. Rushmeier i V. Deschaintre, „Node Graph Optimization Using Differentiable Proxies," u *ACM SIGGRAPH 2022 Conference Proceedings*, 2022, doi: 10.1145/3528233.3530733.

[4] A. Lagae, S. Lefebvre, G. Drettakis i P. Dutré, „Procedural Noise using Sparse Gabor Convolution," *ACM Transactions on Graphics*, vol. 28, br. 3, čl. 54, 2009, doi: 10.1145/1531326.1531360.

[5] R. L. Cook i T. DeRose, „Wavelet Noise," *ACM Transactions on Graphics*, vol. 24, br. 3, str. 803–811, 2005, doi: 10.1145/1073204.1073264.

[6] J. C. Hart, „Perlin Noise Pixel Shaders," u *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Workshop on Graphics Hardware*, 2001, doi: 10.1145/383507.383531.

[7] W. Cai, R. Shea, C.-Y. Huang, K.-T. Chen, J. Liu, V. C. M. Leung i C.-H. Hsu, „A Survey on Cloud Gaming: Future of Computer Games," *IEEE Access*, vol. 4, str. 7605–7620, 2016, doi: 10.1109/ACCESS.2016.2590500.

[8] A. Shehabi et al., *2024 United States Data Center Energy Usage Report*. Berkeley, CA, SAD: Lawrence Berkeley National Laboratory, 2024, doi: 10.71468/P1WC7Q.

[9] AMD GPUOpen, „AMD Compressonator," 2024. [Online]. Dostupno: https://gpuopen.com/compressonator/. [Pristupljeno: 19. oktobar 2026].

[10] J. T. Meech, V. Tsoutsouras i P. Stanley-Marbell, „The Data Movement Bottleneck: Theoretical Shortcomings of Analog Optical Fourier Transform and Convolution Computing Accelerators," arXiv:2308.01719, 2023, doi: 10.48550/arXiv.2308.01719.

[11] S. Lam, A. Khaled, S. Bilodeau, B. A. Marquez, P. R. Prucnal, L. Chrostowski, B. J. Shastri i S. Shekhar, „Neuromorphic Photonic Computing with an Electro-Optic Analog Memory," *Nature Communications*, vol. 17, čl. 2472, 2026, doi: 10.1038/s41467-026-69084-x.

[12] J. M. P. van Waveren, „id Tech 5 Challenges: From Texture Virtualization to Massive Parallelization," prezentacija, id Software, 2009.